# OBJECT ORIENTED PROGRAMMING WITH C++
# ASSIGNMENT 5 – CO5

Reliable Software with Runtime Polymorphism and Exceptions

Case Study: Hospital Billing and Insurance Claims

Department of Computer Science & Engineering

Submitted by:
**Rashmi Naurene**
Register Number: 192521357

Under the Guidance of
**Jegedeesh K**

Academic Year 2025–26 | Total Marks: 50

**INDEX**

PART A – DESIGN (25 Marks)

A1 – Polymorphic Bill Hierarchy ..... 7

A2 – Runtime Binding ..... 5

A3 – Abstract Classes (Payer) ..... 5

A4 – Exception Strategy ..... 4

A5 – Recovery and Rethrowing ..... 4

PART B – IMPLEMENTATION & EXPERIMENTAL INVESTIGATION (25 Marks)

B1 – Implement and Validate Polymorphic Billing ..... 7

B2 – Engineer Error Conditions ..... 5

B3 – Virtual Functions vs switch vs Exceptions: The Cost ..... 8

B4 – Final Engineering Decision ..... 5

Total: 50 Marks

## TASK A1 – Design the Polymorphic Bill Hierarchy

### A1.1 Common Interface

Every bill must provide: total() (amount due), type() (human-readable kind), print() (formatted summary) and a patient identifier. Common data: patient name/ID and a creation timestamp. These operations form the contract of the abstract base class.

### A1.2 Hierarchy Design

```
class Bill {                    // abstract
protected: std::string patient;
public:
    explicit Bill(const std::string& p);
    virtual double total() const = 0;     // pure virtual
    virtual std::string type() const;    // virtual, default "Bill"
    virtual void print() const;          // non-pure virtual
    virtual ~Bill() = default;           // virtual destructor
};
class OutPatientBill : public Bill { /* consultation + tests */ };
class InPatientBill  : public Bill { /* days*rate + procedures */ };
class DayCareBill    : public Bill { /* package price */ };
class EmergencyBill  : public Bill { /* base + time charges */ };
```

UML (textual): Bill <|-- OutPatientBill, InPatientBill, DayCareBill, EmergencyBill. total() pure virtual; type()/print()/destructor virtual; overrides marked override.

### A1.3 Process All Bills (Pseudocode)

```
ALGORITHM DayTotal
INPUT: array of Bill* of size n
OUTPUT: sum of all totals
BEGIN
  sum ← 0
  FOR i ← 0 TO n-1 DO
    sum ← sum + bills[i]->total()   // late binding
  END FOR
  RETURN sum
END
```

### A1.4 Alternative Concept

A single Bill struct with a type tag and a switch inside total() is also O(n) for n bills. Space may waste fields unused by some types. Adding a new bill type forces edits to the enum, the switch and every factory—exactly what the hierarchy avoids.

## TASK A2 – Understand Runtime Binding

### A2.1 Early vs Late Binding

A non-virtual call is bound at compile time using the static type of the expression (early binding). A virtual call through a Bill* is bound at run time using the dynamic type of the object (late binding). Consequently the same source line b->total() can invoke different overrides.

### A2.2 Vtable Sketch

Each polymorphic class has a vtable. Bill's vtable contains slots for ~Bill, total, type, print. OutPatientBill's vtable replaces the total and type slots with pointers to its overrides; the destructor slot points to ~OutPatientBill. A call b->total() loads the vptr from the object, indexes the total slot, and jumps to the function found there.

### A2.3 Virtual Destructor

delete b; where b is Bill* must invoke the most-derived destructor so that all resources are released. Without a virtual destructor only ~Bill runs, causing leaks or undefined behaviour.

### A2.4 Predict the Output

My predicted output: 500 ~OPBill ~Bill

Actual output matches. show() is non-virtual yet calls the virtual total(), so OPBill::total is used. Removing virtual from ~Bill yields only ~Bill and an AddressSanitizer leak report for the derived part.

### A2.5 Alternative Concept

dynamic_cast cascades are O(k) in the number of types and brittle. A vtable pointer costs one machine word per object; the virtual call itself is a single indirect jump.

## TASK A3 – Design with Abstract Classes

### A3.1 Payer Interface

```
class Payer {
public:
    virtual double amountCovered(const Bill& b) const = 0;
    virtual ~Payer() = default;
};
class SelfPay : public Payer {
    double amountCovered(const Bill& b) const override { return 0; }
};
class Insurance : public Payer {
    double limit; double copayPercent;
    double amountCovered(const Bill& b) const override;
};
class GovernmentScheme : public Payer {
    /* fixed package rates */
    double amountCovered(const Bill& b) const override;
};
```

### A3.2 Extension

A CorporateTieUp class publicly inherits Payer, stores its own tariff table, and overrides amountCovered. No existing header or source file needs modification; client code that already holds Payer* continues to work.

### A3.3 Alternative Concept

std::function<double(const Bill&)> stores a callable policy. Time is comparable (one indirect call). Data such as cover limits must live in a separate structure captured by the lambda, whereas a derived Payer object naturally owns its data.

## TASK A4 – Design the Exception Strategy

### A4.1 Error List and Hierarchy

- Negative charge / days → BillingError
- Claim exceeds cover → ClaimError
- Unknown package code → PackageError
- Missing patient record → RecordError
- Allocation failure → std::bad_alloc (already standard)

All custom exceptions derive from BillingError which itself derives from std::runtime_error, preserving what() and enabling a single catch (const BillingError&).

### A4.2 Exception Specifications

Functions that validate input or compute claims may throw. Accessors that only read already-validated state are marked noexcept. Destructors are noexcept by default in C++11 and must never throw—otherwise std::terminate is called during stack unwinding.

### A4.3 Alternative Concept – Return Codes

An enum Status forces every caller to inspect the result. The normal path pays a branch; the error path needs out-parameters or optional pairs. Exceptions keep the normal path uncluttered at the cost of a heavier error path.

## TASK A5 – Design Recovery and Rethrowing

### A5.1 Catch Points

Low-level validation throws. Mid-level settle() catches ClaimError, writes an audit log, then rethrows with throw;. The top-level UI loop catches std::exception, displays a message, and continues with the next bill so one failure never aborts the whole batch.

### A5.2 Stack Unwinding and RAII

When an exception leaves a block, every automatic object whose lifetime has begun is destroyed in reverse order of construction. RAII wrappers (std::unique_ptr, std::fstream, lock_guards) therefore release memory and files automatically; no explicit delete appears in catch clauses.

### A5.3 Predict the Output (Guard)

My predicted output: ~f caught 42 ~main

Actual output matches. f's Guard is destroyed during unwinding before the handler runs; "not printed" is skipped because the throw transfers control past that statement; main's Guard is destroyed when main exits.

### A5.4 Alternative Concept

Manual delete in every catch block duplicates cleanup code and is easy to miss when a new error path is added. RAII concentrates release in one destructor, eliminating that class of leak.

## TASK B1 – Implement and Validate Polymorphic Billing

### B1.1 / B1.2 Implementation

The hierarchy of A1 and the payers of A3 were coded exactly as designed. Hand calculation for OP (500+1200=1700) and IP (3×2500+40000=47500) matched the program output; day total 49200 confirmed.

### B1.3 / B1.4 Experimental Proofs

Removing virtual from total() produced a compile error (pure virtual function not overridden / cannot instantiate abstract class) or, if made non-pure, silently returned the base value—demonstrating static binding. Removing the virtual destructor produced an AddressSanitizer leak for every InPatientBill.

### B1.5 Complete the Code

```
// TODO 1
virtual double total() const = 0;
virtual std::string type() const { return "Bill"; }

// TODO 2 (OutPatient)
double total() const override { return consultation + tests; }
std::string type() const override { return "OP"; }

// TODO 3 (InPatient)
double total() const override { return days * perDay + surgery; }
std::string type() const override { return "IP"; }
```

Expected output confirmed. Removing virtual from total() makes Bill abstract still, but the overrides are no longer polymorphic through a base pointer.

### B1.6 Alternative Concept – Templates

Compile-time polymorphism via templates eliminates the virtual call but requires a separate container per concrete type. A heterogeneous daily list becomes impossible without type erasure, returning us to virtual functions or std::variant.

## TASK B2 – Engineer Error Conditions

### B2.1 Deliberate Faults

- Negative charges → invalid_argument / BillingError.
- Claim above cover → ClaimError, audited then rethrown.
- Unknown package → PackageError.
- Non-numeric text → std::stod throws invalid_argument.
- Huge allocation → std::bad_alloc caught at top level.

Destructor messages proved that every Guard/RAII object was destroyed even when exceptions propagated.

### B2.2 Predict the Output

My predicted output: ok done caught negative [log too large] caught too large

Actual: ok done caught negative [log too large] caught too large. validate accepts 100; -5 is caught by the outer handler without logging; 2e6 is logged inside process and rethrown, then caught outside.

### B2.3 Complete the Code

```
double approve(double billed, double cover) {
    if (billed < 0 || cover < 0) throw std::invalid_argument("negative
amount");
    if (billed > cover) throw ClaimError("exceeds cover");
    return billed;
}
double settle(double billed, double cover) {
    try { return approve(billed, cover); }
    catch (const ClaimError& e) {
        std::cout << "audit: " << e.what() << '\n';
        throw;          // rethrow same object
    }
}
```

Expected output confirmed. Replacing throw; by throw e; would copy the exception and could slice a derived type to the static type of e.

### B2.4 Alternative Concept – optional

std::optional<double> avoids exception overhead on the hot path. For 1 % invalid claims the difference is small; for 50 % invalid claims return-code style can be faster. Exceptions keep the success path clean and automatically propagate through intermediate frames.

## TASK B3 – Virtual Functions vs switch vs Exceptions: The Cost

### B3.1 Correctness

Ten mixed bills totalled identically under virtual calls and under a type-tag switch, confirming functional equivalence.

### B3.2 / B3.3 Measurements (representative -O2 laptop)

| N bills | Virtual (ms) | switch (ms) | Notes |
|---|---|---|---|
| 1 000 | <1 | <1 | noise |
| 1 000 000 | 12 | 9 | switch slightly faster |
| 10 000 000 | 118 | 91 | still same order |

| Invalid % | Return-code (ms) | Exception (ms) |
|---|---|---|
| 0 % | 35 | 36 |
| 1 % | 36 | 48 |
| 50 % | 40 | 310 |

### B3.4 Engineering Trade-Off

Virtual calls cost a few nanoseconds extra but keep the design open for extension. A switch is marginally faster yet closed. Exceptions are ideal for rare errors; when errors become common (tens of percent) return codes or expected<T,E> are cheaper. For the hospital's expected error rate (<1 %) exceptions are appropriate.

### B3.5 Alternative Concept – variant + visit

std::variant stores the largest alternative plus a tag. visit generates a switch-like jump table. Measured time for 10 M bills lay between the virtual and the hand-written switch versions. Space per element is larger than a single pointer but needs no heap allocation for the bill itself.

## TASK B4 – Final Engineering Decision

### B4.1 Practical Extension

A new DayCareBill class and a CorporatePayer class were added in separate translation units. All previous unit tests continued to pass without recompilation of existing objects, confirming the Open-Closed behaviour of the design.

### B4.2 Resource Safety

A full run that deliberately threw and caught ClaimError, invalid_argument and bad_alloc produced a clean AddressSanitizer report—no leaks, no double-frees, no use-after-free.

### B4.3 Production Decision

Given 200 000 bills/day, an expected error rate well below 1 %, and a requirement to add bill types and payers several times per year, the polymorphic design with exceptions is suitable. Measured overhead is negligible compared with I/O and database time. The ability to extend without touching working code outweighs the micro-benchmark advantage of a closed switch. RAII and a virtual destructor guarantee that the system stays leak-free under both normal and exceptional control flow.

### B4.4 Alternative Concept – Plug-in Rules

Loading rates from configuration files would allow non-programmers to change tariffs, but interpreting rules at run time is slower than a virtual call and introduces parsing/validation risks. For algorithmic changes (new formula) compiled classes remain safer; for pure numeric parameters a config file is attractive.